

Design Space Exploration of a Single-Head Attention Accelerator

Cheng-Hsiu Hsieh

University of California, Los Angeles
Los Angeles, CA, USA
alanhsiu@ucla.edu

Ching-Yu Cheng

University of California, Los Angeles
Los Angeles, CA, USA
ccyu906@g.ucla.edu

Anqi Yang

University of California, Los Angeles
Los Angeles, CA, USA
anqiyang02@g.ucla.edu

Chi-Yu Chen

University of California, Los Angeles
Los Angeles, CA, USA
chiyuchen@ucla.edu

Abstract

GPU roofline analysis of single-head scaled dot-product attention ($D = 64$) on the NVIDIA TITAN V shows the workload is neither compute-bound nor HBM-bandwidth-bound: SM utilization never exceeds 38% and achieved arithmetic intensity falls below the FP16 tensor-core ridge, leaving substantial headroom for a specialized fixed-function datapath.

Motivated by this observation, we present a design space exploration of a single-head INT8 attention accelerator implemented as a configurable systolic array in synthesizable SystemVerilog. We investigate two axes of design freedom: array size scaling (4×4 , 8×8 , 16×16) and PE partitioning ($1 \times 8 \times 8$, $4 \times 4 \times 4$, $16 \times 2 \times 2$) with a fixed total of 64 PEs, synthesized on a TSMC 0.13 μm standard-cell library at 100 MHz.

Among the partitioning strategies, the $16 \times 2 \times 2$ design reduces hardware cycles by 23.6% relative to the $1 \times 8 \times 8$ baseline (458 to 350 cycles) while increasing total area by only 8.1% and leaving power virtually unchanged ($< 0.3\%$). However, the softmax phase — fixed at 176 cycles across all three designs — emerges as the dominant bottleneck, accounting for 50.3% of total runtime in the fastest configuration.

Keywords

attention accelerator, systolic array, GPU baseline, roofline, energy efficiency

1 Introduction

The attention mechanism [6] has become the cornerstone of modern deep learning, powering large language models and vision transformers that have achieved state-of-the-art results across a wide range of tasks. At its core, scaled dot-product attention requires two matrix multiplications and a softmax operation, with computational complexity that scales quadratically with sequence length. As sequence lengths grow to tens of thousands of tokens in contemporary workloads, the cost of attention dominates end-to-end inference latency, motivating the design of dedicated hardware accelerators. General-purpose GPUs, while flexible, are ill-suited for the single-head, low-batch-size attention regime that characterizes prefill inference. The compute-to-memory ratio of attention is substantially lower than that of dense linear layers, leaving GPU tensor cores significantly underutilized. Even with IO-aware kernels such as FlashAttention, which tile the computation to keep intermediate scores on-chip, the workload remains data-movement limited rather

than compute-limited for small head dimensions. Systolic arrays [5] offer a compelling alternative: their regular, pipelined dataflow naturally matches the structure of matrix multiplication, achieving high PE utilization with minimal control overhead and low memory traffic through operand reuse. However, the design space for a systolic-array-based attention accelerator is large. Key choices include array size, the number of parallel sub-arrays, and the partitioning of a fixed PE budget across multiple smaller arrays to reduce wavefront latency. In this work, we present a design space exploration of a single-head INT8 attention accelerator implemented as a configurable systolic array in synthesizable SystemVerilog. We investigate two axes of design freedom: array size scaling (4×4 , 8×8 , 16×16) and PE partitioning ($1 \times 8 \times 8$, $4 \times 4 \times 4$, $16 \times 2 \times 2$) with a fixed total of 64 PEs. We characterize each configuration using a cycle-accurate analytical model anchored to Synopsys Design Vision synthesis results on a 0.13 μm standard-cell library at 100 MHz, and compare against a measured NVIDIA TITAN V GPU baseline. Our analysis reveals that PE partitioning reduces total hardware cycles by up to 23.6% with only an 8.1% area overhead, while softmax emerges as the dominant bottleneck across all configurations, accounting for up to 50.3% of total runtime.¹

2 Related Work

2.1 GPU Attention Kernels

On NVIDIA Volta GPUs, PyTorch’s scaled dot-product attention (SDPA) dispatches to a memory-efficient, FlashAttention-style kernel [2]. Rather than materializing the full $S \times S$ attention matrix in DRAM, this kernel tiles the computation so that intermediate scores remain on chip, trading additional recomputation for greatly reduced memory traffic. This IO-aware strategy is the reason a general-purpose GPU can execute attention at all competitively, and it directly shapes the memory-traffic and utilization behavior we report for our GPU baseline.

2.2 Attention and Systolic-Array Accelerators

Dedicated matrix-multiplication hardware has long underpinned deep-learning acceleration. Systolic arrays, first articulated by Kung [5], form the compute fabric of commercial accelerators such as Google’s TPU [4], whose large array maps dense matmul onto a regular, operand-reusing dataflow—the same property our design exploits at a far smaller scale. Spatial CNN accelerators such as Eyeriss [1] instead optimize the dataflow to minimize off-chip data movement, a

¹Source code available at https://github.com/Alanhsiu/CS259_Final_Project.

concern that also motivates keeping Q , K , and V resident in on-chip buffers in our design.

A second line of work targets the attention operator directly. A^3 [3] and SpAtten [7] accelerate attention by exploiting approximation and token/head sparsity, respectively, pruning the score matrix before it is fully materialized; both attack the quadratic cost of attention *algorithmically*. Our work is complementary and deliberately narrower: rather than altering the attention computation, we hold a dense INT8 datapath fixed and ask how a fixed budget of 64 PEs should be *organized*—a single large array versus several smaller parallel sub-arrays—and where the resulting cycles are actually spent. In contrast to prior attention accelerators, which report end-to-end throughput, our phase-level cycle breakdown isolates the fixed-cost softmax stage as a first-order architectural bottleneck.

3 Methods

3.1 Test Vector Generation

To validate the RTL systolic array independently of the full pipeline, we generate two families of INT8 matrix-multiply test vectors.

Synthetic tiles. For each array size $N \in \{4, 8, 16\}$ we generate ten random INT8 tile pairs (elements drawn uniformly from $[-128, 127]$ with a fixed seed for reproducibility) plus three edge cases: all-zeros, identity, and all-127 (an overflow stress test). The reference product $C = A \cdot B$ is computed in INT32 and all matrices are stored as row-major hex files in the format consumed by the RTL testbench’s \$readmemh. After generation we read each tile back, recompute the product, and assert equality, yielding a per-size PASS/FAIL check.

Real attention tile. To strengthen validation beyond synthetic data, we extract a real attention tile from GPT-2 small. A forward hook on the first attention layer captures the Q and K projections; we take head 0, tokens 0–7, quantize the resulting 8×64 tiles to INT8 using per-tensor symmetric quantization, and store them in the same hex format together with the INT32 reference for QK^T . Per-tensor quantization yields a maximum absolute error of 0.21 and a mean absolute error of 0.066 relative to the FP32 reference. Because the head dimension is 64 while the array processes 8×8 tiles, a full QK^T decomposes into eight partial-sum tiles; the provided metadata documents this decomposition for the RTL team.

3.2 Systolic Array Size Scaling:

$$4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16$$

The PE grid is generated by a parameterized generate loop, so the same RTL instantiates a 4×4 , 8×8 , or 16×16 array by changing a single parameter N . This section characterizes how area and latency evolve as N doubles.

PE count and wavefront latency. An $N \times N$ array contains N^2 PEs. Because inputs are fed with a time-skewed wavefront (Section 4.3), $PE[r][c]$ first receives data at cycle $t = r + c$ and completes its last accumulation (inner dimension K) at $t = r + c + K - 1$. The entire $N \times N$ result is available after the corner PE finishes:

$$t_{\text{comp}} = 2(N - 1) + (K - 1). \quad (1)$$

Because t_{comp} grows *linearly* with N while PE count grows *quadratically*, PE utilization (fraction of PE-cycles that perform useful MACs) *decreases* as the array grows:

$$\eta = \frac{K}{2N + K - 2}. \quad (2)$$

Area scaling. Standalone array synthesis (INT8, 0.13 μm) shows that cell area scales super-quadratically with N : the $4 \times$ increase in PEs from 4×4 to 8×8 raises cell area by $\approx 4.5 \times$, reflecting longer data buses and denser routing. Table 1 summarizes all three sizes for $K=8$.

Table 1: Standalone systolic array scaling ($K=8$, 0.13 μm).

Array	PEs	t_{comp}	PE util. η	Cell area (μm^2)	Total area (μm^2)
4×4	16	13 cy	57.1%	70,065	493,754
8×8	64	21 cy	36.4%	312,549	2,281,006
16×16	256	37 cy	21.1%	1,217,738	9,115,738

[†]Extrapolated from $4 \times 4/8 \times 8$ synthesis trend.

The utilization–latency trade-off. A 16×16 array delivers 256 parallel MACs per active cycle but wastes 78.9% of its PE-cycles on wavefront startup overhead. Conversely, a 4×4 array has the shortest startup overhead (13 cycles) but only 16 concurrent MACs. The 8×8 array is the natural match for the target workload (seq_len=8, output tile 8×8): every PE maps exactly to one output element, and no hardware is left idle waiting for operands at the array boundary.

Fitting the workload. A 16×16 array is over-provisioned for an 8×8 output: only the top-left 8×8 quadrant of PEs would produce valid results, wasting $4 \times$ the silicon. A 4×4 array under-covers the output and requires four passes (or four parallel instances) to fill all 64 output elements. The 8×8 array therefore achieves the best area utilization for this problem size; further cycle reduction is obtained not by growing the array but by *partitioning* it, as described in Section 4.4.

4 Methodology

4.1 GPU Baseline Methodology

Platform. All GPU measurements were collected on a single NVIDIA TITAN V (Volta, GV100, sm_70) with 12 GB of HBM2, using PyTorch 2.6.0 with CUDA 12.4. The hardware parameters used for the roofline model are summarized in Table 2. We treat the GPU as a reference point for general-purpose hardware on this workload rather than as a same-process competitor to the accelerator; process-technology differences make a direct energy comparison inappropriate, so we instead characterize *where* the GPU’s time and energy go.

Workload. The workload is single-head scaled dot-product attention in the prefill setting, batch size 1, head dimension $D = 64$, FP16 precision, executed via `torch.nn.functional.scaled_dot_product_attention`. The sequence length S is swept over $\{1024, 2048, 4096, 8192, 16384\}$. We count the two matrix multiplications as $4S^2D$ FLOPs and exclude the softmax, following the usual convention.

Table 2: NVIDIA TITAN V parameters used for the baseline and roofline.

Parameter	Value
Architecture	Volta (GV100, sm_70)
HBM2 peak bandwidth	652.8 GB/s
FP32 peak	14.9 TFLOPS
FP16 tensor-core peak	110 TFLOPS
Streaming multiprocessors	80
L2 cache	4.5 MB
Thermal design power	~250 W

Latency and throughput. For each S the kernel runs 100 timed iterations after 100 warm-up iterations, each bracketed by `torch.cuda.synchronize()` and timed with `time.perf_counter()`. We report mean and standard deviation and define sustained throughput as $4S^2D$ divided by mean latency.

Memory traffic. Because Nsight Compute replays each kernel to read hardware counters, we profile a single forward pass per S . We collect global load/store traffic between the L2 and L1 caches (`l1tex__m_xbar2l1tex_read_bytes_mem_lg_op_ld.sum` and the corresponding store metric), SM throughput, and DRAM throughput. We report $L1 \leftrightarrow L2$ traffic rather than `dram_bytes` because for the smaller sequence lengths the working set is cache-resident and DRAM writes are effectively zero, which makes a DRAM-based arithmetic intensity ill-defined. Achieved arithmetic intensity is FLOPs divided by total $L1 \leftrightarrow L2$ bytes.

Power and energy. GPU power was logged at 100 ms intervals with `nvidia-smi` while the kernel ran in a tight loop for approximately ten seconds per S . Samples whose timestamps fall in each config’s active window are averaged to obtain mean power; energy per inference is $E = P_{avg} \cdot t_{lat}$. The GPU was shared with other users, so readings include an idle baseline of ~36 W; reported power is therefore a conservative (slightly pessimistic) estimate of the kernel’s own draw.

4.2 RTL Methodology

Systolic array. To accelerate the matrix multiplication operation in the attention mechanism, a systolic array architecture was adopted due to its high parallelism, regular structure, and efficient data reuse. The proposed design consists of configurable 4×4 , 8×8 , and 16×16 INT8 systolic arrays. Each Processing Element (PE) performs a multiply-accumulate (MAC) operation and stores intermediate partial sums locally. Input matrix A is streamed horizontally while matrix B is streamed vertically. Data propagates through neighboring PEs in a pipelined fashion, allowing all MAC units to operate concurrently after the initial filling stage. Compared with a conventional matrix multiplication implementation, the systolic array significantly reduces memory accesses by reusing operands as they move through the array.

As shown in Figure 1, the RTL architecture consists of four major modules: an input buffer, a systolic array, an output buffer, and a controller finite-state machine (FSM). The input buffer stores incoming matrix elements and feeds data into the systolic array according to the computation schedule. The systolic array contains

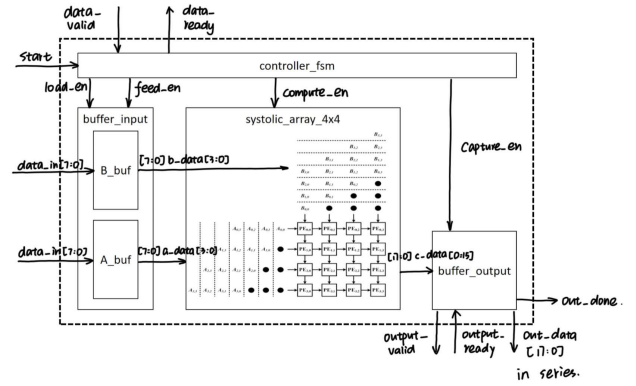
a two-dimensional grid of PEs, where each PE performs one MAC operation per cycle and forwards operands to neighboring PEs. The output buffer captures the final accumulated results and serializes them for output transmission.

Each PE contains operand registers, an INT8 multiplier, an accumulator register, and forwarding logic. At every clock cycle, the PE computes

$$P_{new} = P_{old} + A \times B \quad (3)$$

where P_{old} represents the locally stored partial sum. The updated result is accumulated while operands continue propagating through the array for subsequent computations.

The overall execution flow is controlled by an FSM consisting of five states: IDLE, LOAD, CLEAR, COMPUTE, and OUTPUT. The controller generates control signals such as `load_en`, `feed_en`, `compute_en`, and `capture_en` to coordinate data loading, computation, and result collection. Furthermore, the RTL implementation is fully parameterized, enabling the same codebase to generate 4×4 , 8×8 , and 16×16 systolic arrays for architectural exploration of performance, area, and power trade-offs.

**Figure 1: Top-level RTL architecture of the systolic array.**

Attention kernel. To support end-to-end attention computation, the proposed design extends the systolic-array-based matrix multiplication engine into a complete single-head self-attention accelerator. The module implements

$$Out = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (4)$$

where Q , K , and V are stored in on-chip buffers in INT8 format. To minimize hardware overhead, separate systolic arrays are not instantiated for the two matrix multiplication stages. Instead, a single 8×8 systolic array is time-multiplexed under the control of the attention FSM. The accelerator first computes QK^T and stores the accumulated scores in an on-chip register file. A row-wise softmax operation is then applied to each score row. The resulting attention weights are quantized and fed back into the same systolic array to perform the final matrix multiplication with V , producing the attention output.

The overall execution flow is managed by the `attention_fsm`, which coordinates multiple computation stages including matrix

loading, the first matrix multiplication, score extraction, row-wise softmax, the second matrix multiplication, and result streaming. To avoid the area overhead of a dedicated divider, the scaling factor $1/\sqrt{d_k}$ is approximated using a right-shift operation. The softmax unit is implemented entirely in fixed-point arithmetic. For numerical stability, row-wise maximum subtraction is performed prior to exponentiation. The exponential function is approximated using a lookup table (LUT), and normalization is completed using a restoring divider. Finally, the softmax outputs are converted to Q0.6 fixed-point format before being fed back into the systolic array, ensuring compatibility with the signed arithmetic and accumulation logic of the processing elements.

System. The analytical model takes cycle counts and synthesis PPA figures as inputs and derives system-level efficiency metrics. The cycle model is parameterized by array side length N :

$$\begin{aligned} t_{\text{compute}} &= 2N + 6 \quad (\text{wavefront latency, cycles}) \\ t_{\text{phase}} &= 1 + t_{\text{compute}} + 1 \quad (\text{CLR} + \text{COMP} + \text{CAP}) \\ t_{\text{QK}} &= 8 \times t_{\text{phase}} \quad (\text{NTILES} = 8) \\ t_{\text{softmax}} &= 176 \quad (\text{fixed; shared across all designs}) \\ t_{\text{AttnV}} &= t_{\text{phase}} \\ t_{\text{stream}} &= 65 \\ t_{\text{total}} &= t_{\text{QK}} + 1 + t_{\text{softmax}} + t_{\text{AttnV}} + t_{\text{stream}}. \end{aligned}$$

Two composite efficiency metrics are computed:

$$\begin{aligned} \text{area-latency product} &= t_{\text{total}} \times A_{\text{total}} \quad (\text{lower is better}) \\ \text{energy proxy} &= t_{\text{total}} \times P_{\text{total}} \quad (\text{lower is better}). \end{aligned}$$

Because the final RTL report already provides FSM-level cycle counts, this model is used as a cycle-accounting and PPA analysis tool rather than an independent RTL predictor. The Python implementation reproduces the reported FSM cycle breakdown, then derives latency, area-latency product, and an energy-proxy metric for the three PE partitioning strategies. The energy proxy is proportional to energy per attention invocation because all designs run at the same 100 MHz frequency and have nearly identical power.

4.3 Optimized PE: Removing the Compute-Enable Gate

The baseline PE (pe.sv) includes a `compute_en` signal that gates the accumulator: the MAC updates only when `compute_en` is asserted, leaving the accumulator frozen otherwise. An optimized variant removes this signal entirely, so the accumulator runs unconditionally every clock cycle:

$$\text{acc} += \text{a_in} \times \text{b_in} \quad (\text{every cycle, no enable gate}). \quad (5)$$

Correctness. Correctness is preserved because the top-level feed logic drives `a_data` and `b_data` to zero whenever `feed_en` is de-asserted. A PE receiving zero inputs accumulates $0 \times 0 = 0$, which is an arithmetic no-op. The `clear_acc` signal still resets the accumulator at the start of each tile, so stale state never contaminates a new computation.

Area reduction. Removing `compute_en` eliminates the clock-enable mux on every accumulator flip-flop and removes the control-signal routing to all N^2 PEs. For a 64-PE (8×8) array, this pruning reduces

the combinational cell count and shortens critical paths in the PE grid.

Power increase. Without the enable gate, every PE toggles its multiplier and accumulator outputs every cycle — including during the wavefront startup cycles, between tiles, and during the softmax phase. This idle switching raises dynamic power relative to the gated design.

Trade-off summary. Removing `compute_en` trades a modest *area saving* for higher *dynamic power*. The right choice depends on the operating regime: a design that spends most of its time in compute (high duty cycle, e.g. large batch sizes) benefits from the area reduction; a design with long idle intervals between computations is better served by the gated version to suppress unnecessary switching.

4.4 RTL Implementation

The accelerator is implemented in synthesizable SystemVerilog and synthesized with Synopsys Design Vision targeting a 0.13 μm standard-cell library at 100 MHz.

PE module. Each PE (pe.sv) computes `acc += a_in * b_in` when `compute_en` is asserted. The operand registers `a_out` and `b_out` update *unconditionally* every cycle so that operands already in the pipeline continue propagating after `feed_en` de-asserts, allowing the last-fed data to reach the corner PE at `COMP_DONE` without any additional stall logic. The accumulator is cleared synchronously by `clear_acc` at the start of each tile.

On-chip register files. Three 8×8 -byte buffers (`Q_buf`, `KT_buf`, `V_buf`) hold the input tiles in INT8. A 32-bit `accum_rf` (8×8 words) accumulates partial products across all eight $K=8$ tiles; six extra bits of headroom over the 20-bit single-tile maximum prevent overflow at $K_{\text{total}}=64$. After the full accumulation, `score_rf` stores the right-shifted scores (`accum_rf` $\gg$ 3).

Skewed wavefront feeding. At feed cycle t , the FSM drives

$$\begin{aligned} \text{a_data}[r] &= \text{Q_buf}[r][t-r], \\ \text{b_data}[c] &= \text{KT_buf}[t-c][c], \end{aligned}$$

for valid indices $0 \leq t-r, t-c < N$. This skew ensures that $\text{PE}[r][c]$ receives its $k=0$ operands at cycle $t = r + c$ and accumulates $k = t - r - c$ on each subsequent cycle, completing the tile inner product after K cycles without any bubble cycles between rows or columns. `feed_en` is de-asserted after `FEED_LAST` = $N_{\text{array}} + K - 2$; the data already in the pipeline drains to the corner PE by `COMP_DONE` = $2N_{\text{array}} + K - 3$.

FSM State Alignment vs. Wavefront Latency. As shown in Table 1 and Table 3, the duration of the `MULCOMP` state (t_{phase} boundary) is exactly $t_{\text{comp}} + 1$ cycles. While the final multiply-accumulate (MAC) operation completes combinationaly at the corner PE at cycle t_{comp} , the FSM must hold the compute state for one additional clock cycle. This extra cycle acts as a structural buffer, ensuring that the final pipelined result is safely sampled and locked into the on-chip register file on the subsequent rising clock edge before the FSM transitions to the capture phase.

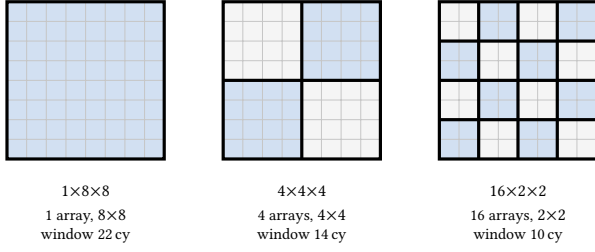


Figure 2: Output-matrix partitioning under a fixed 64-PE budget. The 8×8 output is split into M^2 non-overlapping blocks (shaded), each computed by a dedicated $n \times n$ sub-array with $n = 8/M$. All three configurations use the same 64 PEs; smaller blocks shorten the wavefront path to the corner PE, cutting the per-tile compute window from 22 cycles ($1 \times 8 \times 8$) to 10 cycles ($16 \times 2 \times 2$).

Softmax unit. Row-wise softmax is implemented in fixed-point arithmetic. For each of the eight score rows, a four-stage pipeline subtracts the row maximum (numerical stability), evaluates e^x via a 256-entry LUT, accumulates the exponentials, and divides via a 9-bit restoring reciprocal unit (WRECIP = 9). Total latency per row: $4 + 9 + 8 = 21$ cycles; eight rows run sequentially for $8 \times 22 = 176$ cycles. Outputs are quantized to Q0.6 fixed-point and clamped to $[0, 127]$ before re-entry into the second matmul phase, preventing sign-extension artifacts in the PE signed multiplier.

Controller FSM. A 12-state FSM sequences the full pipeline:

IDLE $\rightarrow$ MUL1CLR $\rightarrow$ MUL1COMP $\rightarrow$ TILECAP ($\times 8$)
 $\rightarrow$ SCORECAP $\rightarrow$ SOFTSTART/SOFTWAIT ($\times 8$)
 $\rightarrow$ MUL2CLR $\rightarrow$ MUL2COMP $\rightarrow$ RESULTCAP $\rightarrow$ STREAM $\rightarrow$ DONE

A single FSM instance drives all three array configurations; only N_ARRAY , $COMP_DONE$, and $FEED_LAST$ differ across variants.

4.5 Sub-Array Partitioning for Reduced Wavefront Latency

The compute window of a single $N_{array} \times N_{array}$ systolic array is determined by how long it takes the wavefront to reach the corner PE. From Eq. (1), this equals $2(N_{array} - 1) + (K - 1)$ cycles. For a fixed $N \times N$ output with inner dimension K , reducing N_{array} by deploying multiple smaller arrays in parallel directly shrinks this window.

Output-block decomposition. The $N \times N$ output matrix C is partitioned into M^2 non-overlapping blocks of size $n \times n$ where $n = N/M$:

$$C[in:(i+1)n, jn:(j+1)n] = A[in:(i+1)n, :] \times B[:, jn:(j+1)n], \quad (6)$$

for $i, j \in \{0, \dots, M-1\}$. Each block is assigned to a dedicated $n \times n$ array $SA[i][j]$. Because every block spans the full K -column extent of A and the full K -row extent of B , all M^2 arrays perform an identical inner product of length K and can run simultaneously under shared control signals (Figure 2).

Input sharing. Arrays in the same block-row i read the same rows of A and share the bus $a_row[i]$; arrays in the same block-column j share $b_col[j]$. This reduces input wiring from $O(NM^2)$ to $O(NM)$.

Local skew delay. Each sub-array uses the *local* index $r' \in \{0, \dots, n-1\}$ for its skew, not the global row index $r = in + r'$. This is correct because the accumulation index inside $SA[i][j]$ is $k = t - r' - c'$, which depends only on the local PE position; the block offset in shifts which row of A is read but does not change the cycle at which the k -th column must arrive at $PE[r'][c']$. At feed cycle t :

$$a_row[i][r'] = Q_buf[in + r'][t - r'],$$

$$b_col[j][c'] = KT_buf[t - c'][jn + c'].$$

Timing comparison. Table 3 quantifies the three configurations illustrated in Figure 2 for $N=8, K=8$.

Table 3: Wavefront timing for three partitioning strategies ($N=8, K=8$).

Configuration	M	n	COMP_DONE	MUL_COMP
$1 \times 8 \times 8$	1	8	$2(7) + 7 = 21$	22 cy
$4 \times 4 \times 4$	2	4	$2(3) + 7 = 13$	14 cy
$16 \times 2 \times 2$	4	2	$2(1) + 7 = 9$	10 cy

Going from $1 \times 8 \times 8$ to $16 \times 2 \times 2$ cuts the per-tile compute window from 22 to 10 cycles (-55%) while keeping total PE count fixed at 64. Over eight tiles, the QK^T phase shrinks from $8 \times 24 = 192$ to $8 \times 12 = 96$ cycles. The softmax (176 cy) and stream (65 cy) phases are unaffected by partitioning, so the overall cycle reduction saturates at -23.6% relative to the $1 \times 8 \times 8$ baseline, as validated in Section 5.2.

5 Evaluation

5.1 GPU Baseline Results

Table 4 reports the full baseline. We organize the analysis around three questions: how performance scales with sequence length, what limits the kernel, and how energy-efficient it is.

Performance scaling. Latency grows sub-quadratically: a $16 \times$ increase in S ($1,024 \rightarrow 16,384$) raises latency only about $23.5 \times$, far below the $256 \times$ expected if the GPU were saturated at every size. The reason is visible in the utilization column: at $S = 1,024$ only 5.1% of SM throughput is used and execution is dominated by launch overhead, while at $S = 8,192$ utilization reaches 38% . Sustained throughput peaks at about 16,250 GFLOPS at $S = 8,192$ — roughly 15% of the FP16 tensor-core peak — before falling back at $S = 16,384$, where $L1 \leftrightarrow L2$ traffic jumps and the working set no longer fits in the 4.5 MB $L2$.

What limits the kernel. The roofline in Figure 3, together with the utilization counters, shows the kernel is neither compute-bound (peak throughput is only $\sim 15\%$ of the tensor-core roof; SM utilization never exceeds 38%) nor HBM-bandwidth-bound (DRAM utilization stays below 1% , and for $S \leq 8,192$ the working set is $L2$ -resident). The achieved arithmetic intensity measured against $L1 \leftrightarrow L2$ traffic ($57\text{--}92$ FLOP/B) sits below the FP16 tensor-core ridge (169 FLOP/B). The binding constraint for this single-head, $D = 64$

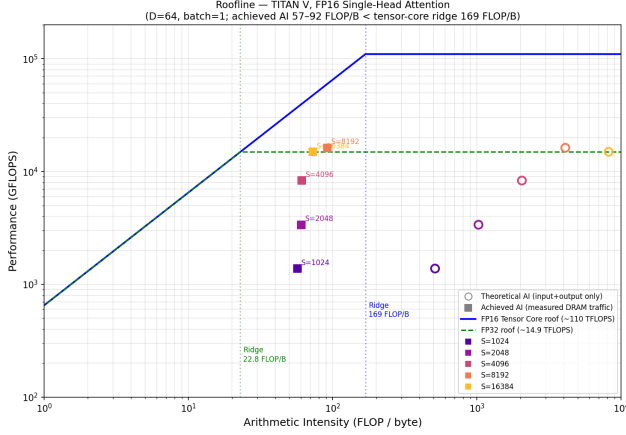


Figure 3: Roofline for TITAN V single-head attention. Open markers: theoretical AI (inputs + output only); filled markers: achieved AI from L1↔L2 traffic. All operating points lie below the tensor-core roof.

shape is thus low SM occupancy — aggravated by launch overhead at small S and by L2 capacity pressure at the largest S — rather than the compute roof or HBM bandwidth. This under-utilized, data-movement-limited regime is precisely what a specialized datapath can exploit.

Energy efficiency. Energy efficiency improves with sequence length, from 29.6 GFLOPS/W at $S = 1,024$ to about 145 GFLOPS/W at $S = 8,192$, because throughput rises much faster than mean power: a lightly loaded GPU still pays most of its static power. Energy per inference rises from 9.1 mJ to 490.7 mJ across the sweep. These figures characterize general-purpose hardware on this workload and serve as context, not as a same-process efficiency comparison.

5.2 RTL Results

5.2.1 Optimization.

Cycle reduction from PE repartitioning. The matmul compute window shrinks as the array side length decreases, because the systolic wavefront travels fewer hops. Table 5 shows the full phase-by-phase breakdown at 100 MHz.

Moving from a single 8×8 array to sixteen 2×2 arrays reduces the total hardware cycle count by 23.6%, from 458 to 350 cycles, with a corresponding latency reduction from 4.58 μ s to 3.50 μ s.

Softmax as the binding bottleneck. The softmax phase is identical across all three designs at 176 cycles, because the number of output rows (8), the softmax pipeline depth (4 stages), and the restoring-divider width (WRECIP = 9) are unchanged by array partitioning. As the matmul phases shrink, softmax grows from 38.4% of total runtime ($1 \times 8 \times 8$) to 50.3% ($16 \times 2 \times 2$), becoming the dominant bottleneck, as visualized in Figure 4. This suggests that further optimization should focus less on repartitioning the systolic arrays and more on the fixed softmax stage, for example by parallelizing softmax across rows or pipelining the divider/LUT path.

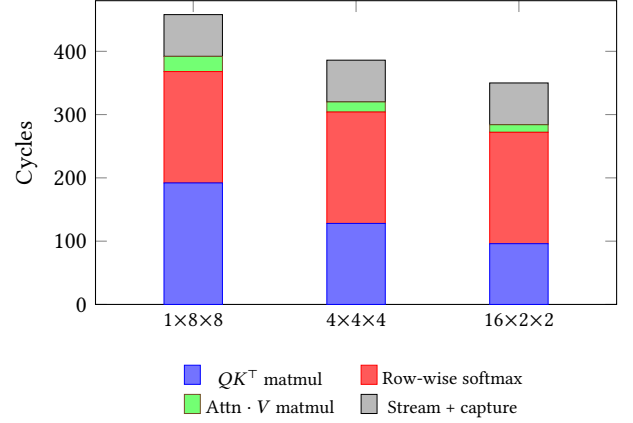


Figure 4: Per-phase cycle breakdown for the three PE-partitioning strategies at 100 MHz. The softmax stage (red) is fixed at 176 cycles regardless of array topology; as the matmul phases shrink with finer partitioning, softmax grows from 38.4% of total runtime in $1 \times 8 \times 8$ to 50.3% in $16 \times 2 \times 2$, becoming the dominant bottleneck.

5.2.2 Comparison.

Synthesis PPA. Table 6 reports synthesis area and power for all three configurations. Sequential cell count is identical across all three designs (9,491 cells), confirming that the register files, FSM, softmax unit, and output buffer are shared and unchanged. All area and power differences originate in the combinational logic and routing.

Total power is within 0.3% across all three designs. Register power ($\approx 90\%$) is determined almost entirely by the shared datapath, making power a non-differentiating factor. The meaningful trade-off is therefore cycle count versus routing area overhead: more arrays require more input multiplexing and inter-array wiring, which increases net interconnect area.

Normalized efficiency. Table 7 reports the two composite cost metrics normalized to the best design in each column.

The $16 \times 2 \times 2$ configuration achieves the best score on both metrics. Its 23.6% cycle reduction more than compensates for the 8.1% total-area overhead, yielding a 17.3% better area-latency product and a 23.5% lower energy proxy than the baseline $1 \times 8 \times 8$ design.

Model consistency check. Since the analytical model is anchored to the FSM cycle breakdown reported by the RTL team, the cycle-count table should be interpreted as a consistency check rather than an independent validation. The model reproduces the reported totals for all three designs, confirming that the Python implementation uses the same cycle accounting before deriving latency, area-latency, and energy-proxy metrics.

6 Conclusions

We have explored two design axes for a single-head INT8 attention accelerator: array size scaling and PE partitioning with fixed total PE count.

Table 4: GPU baseline for single-head FP16 attention ($D=64$, batch=1) on the TITAN V.

S	Latency (ms)	Std (ms)	GFLOPS	L1↔L2 (MB)	Achieved AI (FLOP/B)	SM util (%)	Power (W)	GFLOPS/W
1,024	0.195	0.051	1,380	4.70	57.1	5.1	46.6	29.6
2,048	0.318	0.031	3,376	17.70	60.7	10.4	58.3	57.9
4,096	0.516	0.007	8,322	70.26	61.1	21.0	80.7	103.1
8,192	1.057	0.010	16,250	186.71	92.0	38.0	112.0	145.1
16,384	4.590	0.164	14,973	941.93	73.0	32.9	106.9	140.0

Table 5: Cycle breakdown and latency for the three RTL configurations.

Phase	1×8×8	4×4×4	16×2×2
Matmul compute window	22 cy	14 cy	10 cy
QK^T tiled matmul	192 cy	128 cy	96 cy
Score capture	1 cy	1 cy	1 cy
Row-wise softmax	176 cy	176 cy	176 cy
Attn · V matmul	24 cy	16 cy	12 cy
Stream + done	65 cy	65 cy	65 cy
Total cycles	458	386	350
Latency @ 100 MHz	4.58 μs	3.86 μs	3.50 μs
Reduction vs. 1×8×8	—	−15.7%	−23.6%

Table 6: Synthesis PPA results (Synopsys DC, 0.13 μ m).

Metric	1×8×8	4×4×4	16×2×2
Total cell area (μm^2)	962,640	943,418	972,359
Net interconnect area (μm^2)	6,958,222	7,111,616	7,586,685
Total area (μm^2)	7,920,862	8,055,035	8,559,044
Total power (mW)	23.04	22.98	23.05
of which register	20.77 (90.1%)	20.69 (90.0%)	20.63 (89.5%)
Area vs. 1×8×8	baseline	+1.7%	+8.1%
Power vs. 1×8×8	baseline	−0.25%	+0.07%

Table 7: Normalized area–latency and energy efficiency (lower is better).

Design	Cycles × area	Norm.	Cycles × power	Norm.
1×8×8	3.628×10^9	1.211×	10,552	1.308×
4×4×4	3.109×10^9	1.038×	8,870	1.099×
16×2×2	2.996×10^9	1.000×	8,068	1.000×

Table 8: Model consistency check against RTL report cycle counts.

Design	RTL report	Model accounting	Difference
1×8×8	458 cy	458 cy	0
4×4×4	386 cy	386 cy	0
16×2×2	350 cy	350 cy	0

On the array size axis, PE utilization decreases as the array grows ($N = 4$: 57.1%, $N = 8$: 36.4%, $N = 16$: 21.1%), because the

wavefront startup latency scales linearly with N while PE count scales quadratically. The 8×8 array is the natural match for the $S = 8$, $D = 64$ workload: every PE maps to exactly one output element with no hardware wasted.

On the partitioning axis, splitting the same 64 PEs into smaller sub-arrays reduces the per-tile compute window from 22 cycles ($1 \times 8 \times 8$) to 10 cycles ($16 \times 2 \times 2$), yielding a 23.6% reduction in total hardware cycles at a modest 8.1% area overhead. Because total power varies by less than 0.3% across all three configurations — register power from the shared datapath dominates — the cycle reduction translates directly into energy savings. The $16 \times 2 \times 2$ design achieves the best normalized area–latency product and energy proxy among all configurations evaluated.

In both axes, softmax emerges as the binding bottleneck: it is fixed at 176 cycles regardless of array topology and accounts for up to 50.3% of total runtime. The GPU baseline confirms that the target workload is data-movement limited rather than compute-limited, with SM utilization peaking at 38% and achieved arithmetic intensity well below the FP16 tensor-core ridge. This under-utilized regime is precisely where a specialized fixed-function datapath can exploit regularity to improve efficiency.

Future work should focus on three directions: (1) parallelizing softmax across rows to eliminate the 176-cycle serial bottleneck; (2) widening the on-chip load interface from one byte per cycle to N bytes per cycle to reduce input-skew latency; and (3) extending the design to multi-head attention with shared register files to amortize the fixed datapath overhead across heads.

7 Statement of Work

Cheng-Hsiu Hsieh led and coordinated the project: he proposed the topic, defined the design-space-exploration scope, organized the division of work, and provided ongoing technical guidance to teammates throughout the implementation. His primary contributions were the GPU baseline and RTL test infrastructure: the latency and throughput sweep using PyTorch SDPA; memory-traffic and utilization profiling with Nsight Compute and the achieved arithmetic intensity; sustained-power measurement for energy per inference and GFLOPS/W; the roofline analysis; and the INT8 matrix-multiply test vectors for the 4×4 , 8×8 , and 16×16 arrays, with a quantized real attention tile from GPT-2. He also wrote the Related Work and GPU baseline methodology and evaluation sections, compiled the references, produced the sub-array partitioning and cycle-breakdown figures, and revised sections across the GPU, RTL, and modeling components to ensure consistency.

Anqi Yang was responsible for the RTL design and implementation of the systolic-array accelerator. This included developing

the INT8 systolic-array architecture; designing the processing elements (PEs), input and output buffers, and dataflow mechanisms; implementing the controller FSM for data loading, computation scheduling, and result collection; and writing RTL methodology subsections.

Ching-Yu Cheng was responsible for the analytical performance model and the RTL PPA comparison. This included developing a Python-based cycle-accounting model for the fixed-64-PE partitioning study; deriving latency, area-latency, and energy-proxy metrics from the RTL cycle breakdown and Synopsys DC synthesis results; comparing the $1 \times 8 \times 8$, $4 \times 4 \times 4$, and $16 \times 2 \times 2$ PE partitioning strategies; checking the model for consistency with the RTL report cycle counts; and writing the analytical model methodology, RTL PPA comparison, and model consistency tables.

Chi-Yu Chen was responsible for the RTL design-space exploration and PE-level optimization. This included designing and implementing the four- 4×4 and sixteen- 2×2 parallel systolic array variants by partitioning the 8×8 output matrix into sub-blocks each served by a dedicated parallel sub-array; deriving and applying the local-skew delay principle to ensure correct inner-product accumulation across tiles; conducting the PE optimization study by removing the compute-enable gate and characterizing the resulting area-power trade-off; running synthesis for all three configurations and resolving the inter-run clock-network power methodology discrepancy; constructing the three-way PPA comparison with

normalized area-latency and energy-proxy metrics; and writing the array size scaling, PE optimization, RTL implementation, sub-array partitioning sections, and the RTL optimization subsection, together with all supporting tables.

References

- [1] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 127–138.
- [2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [3] Tae Jun Ham, Sung Jun Jung, Seonghak Kim, Young H. Oh, Yeonhong Park, Jung-Hun Park, Sanghee Lee, Kyoung Park, Jae W. Lee, and Deog-Kyoon Jeong. 2020. A^3 : Accelerating Attention Mechanisms in Neural Networks with Approximation. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 328–341.
- [4] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. 1–12.
- [5] H. T. Kung. 1982. Why systolic architectures? *Computer* 15, 1 (1982), 37–46.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*.
- [7] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 97–110.